

UIL COMPUTER SCIENCE WRITTEN TEST

2026 REGION

APRIL 2026

General Directions (Please read carefully!)

1. DO NOT OPEN THE EXAM UNTIL TOLD TO DO SO.
2. There are 40 questions on this contest exam. You will have 45 minutes to complete this contest.
3. All answers must be legibly written on the answer sheet provided. Indicate your answers in the appropriate blanks provided on the answer sheet. Clean erasures are necessary for accurate grading.
4. You may write on the test packet or any additional scratch paper provided by the contest director, but NOT on the answer sheet, which is reserved for answers only.
5. All questions have ONE and only ONE correct answer. There is a 2-point penalty for all incorrect answers.
6. Tests may not be turned in until 45 minutes have elapsed. If you finish the test before the end of the allotted time, remain at your seat and retain your test until told to do otherwise. You may use this time to check your answers.
7. If you are in the process of actually writing an answer when the signal to stop is given, you may finish writing that answer.
8. All provided code segments are intended to be syntactically correct, unless otherwise stated. You may also assume that any undefined variables are defined as used.
9. A reference to many commonly used Java classes is provided with the test, and you may use this reference sheet during the contest. AFTER THE CONTEST BEGINS, you may detach the reference sheet from the test booklet if you wish.
10. Assume that any necessary import statements for standard Java SE packages and classes (e.g., `java.util`, `System`, etc.) are included in any programs or code segments that refer to methods from these classes and packages.
11. NO CALCULATORS of any kind may be used during this contest.

Scoring

1. Correct answers will receive **6 points**.
2. Incorrect answers will lose **2 points**.
3. Unanswered questions will neither receive nor lose any points.
4. In the event of a tie, the student with the highest percentage of attempted questions correct shall win the tie.

STANDARD CLASSES AND INTERFACES – SUPPLEMENTAL REFERENCE

package java.lang

```
class Object
    boolean equals(Object anotherObject)
    String toString()
    int hashCode()

interface Comparable<T>
    int compareTo(T anotherObject)
        Returns a value < 0 if this is less than anotherObject.
        Returns a value = 0 if this is equal to anotherObject.
        Returns a value > 0 if this is greater than anotherObject.

class Integer implements Comparable<Integer>
    Integer(int value)
    int intValue()
    boolean equals(Object anotherObject)
    String toString()
    String toString(int i, int radix)
    int compareTo(Integer anotherInteger)
    static int parseInt(String s)

class Double implements Comparable<Double>
    Double(double value)
    double doubleValue()
    boolean equals(Object anotherObject)
    String toString()
    int compareTo(Double anotherDouble)
    static double parseDouble(String s)

class String implements Comparable<String>
    int compareTo(String anotherString)
    boolean equals(Object anotherObject)
    int length()
    String substring(int begin)
        Returns substring(begin, length()).
    String substring(int begin, int end)
        Returns the substring from index begin through index (end - 1).
    int indexOf(String str)
        Returns the index within this string of the first occurrence of str. Returns
        -1 if str is not found.
    int indexOf(String str, int fromIndex)
        Returns the index within this string of the first occurrence of str, starting
        the search at fromIndex. Returns -1 if str is not found.
    int indexOf(int ch)
    int indexOf(int ch, int fromIndex)
    char charAt(int index)
    String toLowerCase()
    String toUpperCase()
    String[] split(String regex)
    boolean matches(String regex)
    String replaceAll(String regex, String str)

class Character
    static boolean isDigit(char ch)
    static boolean isLetter(char ch)
    static boolean isLetterOrDigit(char ch)
    static boolean isLowerCase(char ch)
    static boolean isUpperCase(char ch)
    static char toUpperCase(char ch)
    static char toLowerCase(char ch)

class Math
    static int abs(int a)
    static double abs(double a)
    static double pow(double base, double exponent)
    static double sqrt(double a)
    static double ceil(double a)
    static double floor(double a)
    static double min(double a, double b)
    static double max(double a, double b)
    static int min(int a, int b)
    static int max(int a, int b)
    static long round(double a)
    static double random()
        Returns a double greater than or equal to 0.0 and less than 1.0.
```

package java.util

```
interface List<E>
class ArrayList<E> implements List<E>
    boolean add(E item)
    int size()
    Iterator<E> iterator()
    ListIterator<E> listIterator()
    E get(int index)
    E set(int index, E item)
    void add(int index, E item)
    E remove(int index)

class LinkedList<E> implements List<E>, Queue<E>
    void addFirst(E item)
    void addLast(E item)
    E getFirst()
    E getLast()
    E removeFirst()
    E removeLast()

class Stack<E>
    boolean isEmpty()
    E peek()
    E pop()
    E push(E item)

interface Queue<E>
class PriorityQueue<E>
    boolean add(E item)
    boolean isEmpty()
    E peek()
    E remove()

interface Set<E>
class HashSet<E> implements Set<E>
class TreeSet<E> implements Set<E>
    boolean add(E item)
    boolean contains(Object item)
    boolean remove(Object item)
    int size()
    Iterator<E> iterator()
    boolean addAll(Collection<? extends E> c)
    boolean removeAll(Collection<?> c)
    boolean retainAll(Collection<?> c)

interface Map<K,V>
class HashMap<K,V> implements Map<K,V>
class TreeMap<K,V> implements Map<K,V>
    Object put(K key, V value)
    V get(Object key)
    boolean containsKey(Object key)
    int size()
    Set<K> keySet()
    Set<Map.Entry<K, V>> entrySet()

interface Iterator<E>
    boolean hasNext()
    E next()
    void remove()

interface ListIterator<E> extends Iterator<E>
    void add(E item)
    void set(E item)

class Scanner
    Scanner(InputStream source)
    Scanner(String str)
    boolean hasNext()
    boolean hasNextInt()
    boolean hasNextDouble()
    String next()
    int nextInt()
    double nextDouble()
    String nextLine()
    Scanner useDelimiter(String regex)
```

STANDARD CLASSES AND INTERFACES – SUPPLEMENTAL REFERENCE

Package `java.util.function`

```
Interface BiConsumer<T,U>  
    void accept(T t, U u)  
  
Interface BiFunction<T,U,R>  
    R apply(T t, U u)  
  
Interface BiPredicate<T,U>  
    boolean test(T t, U u)  
  
Interface Consumer<T>  
    void accept(T t)  
  
Interface Function<T,R>  
    R apply(T t)  
  
Interface Predicate<T>  
    boolean test(T t)  
  
Interface Supplier<T>  
    T get()
```

UIL COMPUTER SCIENCE WRITTEN TEST – 2026 REGION

Note: Correct responses are based on **Java SE Development Kit 25 (JDK 25)** from Oracle, Inc. All provided code segments are intended to be syntactically correct, unless otherwise stated (e.g., "error" is an answer choice) and any necessary Java SE 25 Standard Packages have been imported. Ignore any typographical errors and assume any undefined variables are defined as used. **For all output statements, assume that the System class has been statically imported using: `import static java.lang.System.*;`**

Question 1

What base would 1212 need to be in to ensure that when converted to base 10, the value is between 60_{10} and 150_{10} ?

- A) Base 3 B) Base 4 C) Base 5 D) Base 6 E) Base 7

Question 2

What is output by the code to the right?

- A) 5.5
B) 6
C) 5
D) 6.0
E) 4.0

```
int a = 6;
double b = 3.0;
int c = 4;

out.println(a * c / b + 10 % c -
            (int) (b + 1.5));
```

Question 3

How many lines of output are generated by the code to the right?

Lines containing only a newline character do not count. The following code produced only one line of output for example.

```
out.println("Hello World");
```

- A) 8 Lines B) 9 Lines
C) 10 Lines D) 11 Lines
E) 12 Lines

```
String s = "hi\nthere";
out.println(s);
out.printf("%s\n%s\n", "one", "two\nthree");
out.print("a\n\nb");
out.println("x\ny");
out.printf("end%s", "\n");
```

Question 4

What is output by the code to the right?

- A) WorldWorldHelloHello
B) HelloHelloHelloHello
C) HelloWorldHelloWorld
D) WorldHelloHelloWorld
E) HelloHelloWorldHello

```
String Hello = "World";
String World = "Hello";
String Hello_World = World.repeat(2) +
                      Hello + "Hello";

out.println(Hello_World);
```

Question 5

What is output by the code to the right?

- A) false
B) true

```
boolean a = true;
boolean b = false;
boolean c = true;

boolean result = (a ^ b) &&
                !(b || c) || (a && c);
out.println(result);
```

Question 6 What is output by the code to the right? A) 7 3.0 B) 7 4.0 C) 4 3.0 D) 7 3.6	<pre>int a = -7; double b = 3.6; out.println(Math.max(Math.abs(a), Math.round(b)) + " " + Math.min(Math.floor(b), Math.ceil(b)));</pre>
Question 7 What is output by the code to the right? A) 5 B) 6 C) 3 D) 7 E) 13	<pre>int a = 6; int b = 3; char c = 'C'; boolean flag = (a & b) > 1; int result = (c & 0x0F) (flag ? a ^ b : a & b); out.println(result);</pre>
Question 8 What is output by the code to the right? A) 501 99 B) 500 100 C) 100 D) 501 E) 501 100	<pre>int x = 100; int y = 500; if (x > 'a' y++ < 501) { out.println(y); } if (-x-- > ~x) { out.println(x); }</pre>
Question 9 What is output by the code to the right? A) 10 B) 12 C) 0 D) There is no output due to a runtime error. E) There is no output due to a compile time error.	<pre>for (int i = -1, total = 0; i < 19; i+=3) { int x = i / (i % 7); total += i; } out.println(total);</pre>
Question 10 What is output by the code to the right? A) [2, 5, 3, 4, 3] B) [1, 3, 3, 4, 3] C) [2, 3, 3, 4, 5] D) [1, 5, 3, 4, 5] E) There is no output due to an error.	<pre>int [] arr = new int[] {1, 2, 3, 4, 5}; arr[0] = arr[arr[0]]; arr[1] += arr[4] = arr[2]; out.println(Arrays.toString(arr));</pre>
Question 11 Which of the following interfaces does Scanner not implement? A) Closeable B) AutoCloseable C) Iterator D) Object E) Scanner implements all of the above.	

Question 12 Which of the following values is output on its own line by the code to the right? A) 899 B) 18 C) 101 D) 50 E) 99	<pre>for (int i = 10; i >= 1; i--) { int product = i * (i - 1); for (int j = 1; j < i; j++) { product += product / j; } out.println(product); }</pre>
Question 13 Which of the following is the correct order of precedence from highest to lowest? A) ++(postfix), ~, *, ==, (bitwise), && B) ~, ++(postfix), *, ==, (bitwise), && C) ++(postfix), ~, *, (bitwise), ==, && D) ++(postfix), *, ~, ==, (bitwise), && E) *, ~, ++(postfix), ==, &&, (bitwise)	
Question 14 Which of the following is the correct order of floating-point values smallest to largest? A) negative infinity, finite negative values, negative zero, zero, finite positive values, infinity B) negative infinity, finite negative values, zero, negative zero, finite positive values, infinity C) Both A and B.	
Question 15 What is the largest possible sum of values in the list? A) 4950 B) 5050 C) 2475 D) 2525 E) 0	<pre>List<Integer> list = new ArrayList<>(); for (int i = 0; i < 100; i++) { list.add(i); int value = (int) (Math.random() * list.size()); Integer t = Integer.valueOf(value); list.remove(t); }</pre>
Question 16 What is output by the code to the right? A) 23 B) 13 C) There is no output due to a compile error. D) There is output, followed by a runtime error. E) There is no output due to a runtime error.	<pre>try { String[] st = new String[5]; Object[] o = st; o[2] = 39.98; out.print(1); } catch (ArrayStoreException e) { out.print(2); } finally { out.print(3); }</pre>

Question 17 What could replace <1*> in the code to the right so that it will compile and run without error (removing items). A) Queue<Integer> B) PriorityQueue<> C) Queue<> D) Either A or B. E) All of the above.	
Question 18 Assuming <1*> is filled in correctly, what is output by the code to the right? A) [5, 6, 7, 8, 9, 10, 12] B) [5, 6, 8, 7, 9, 12, 10] C) [5, 6, 8, 9, 7, 12, 10] D) [2, 3, 4, 5, 6, 9, 10] E) There is no output due to a runtime error.	<pre> <1*> pq; pq = new PriorityQueue<>(); int[] i = new int[] { 12, 8, 7, 3, 9, 5, 10, 2, 4, 6 }; for(int j = 0; j < i.length; j++) pq.add(i[j]); pq.poll(); pq.poll(); pq.poll(); out.println(pq); </pre>
Question 19 If the integer array i was rewritten to instead be of length N, what would be the runtime of the code to the right? A) $\mathcal{O}(N \log N)$ B) $\mathcal{O}(N)$ C) $\mathcal{O}(\log N)$ D) $\mathcal{O}(N^2 \log N)$ E) $\mathcal{O}(N^2)$	
Question 20 Which of the lines in the code to the right first causes an error? A) //1 B) //2 C) //3 D) //4 E) None of the above lines cause any errors.	<pre> List a; //1 a = new ArrayList(10); //2 a.add("Someone"); //3 a.add(a.size(), 43); //4 </pre>
Question 21 What is output by the line marked //q21 in the code to the right? A) 70 B) 0 C) 35 D) 140 E) There is no output due to infinite recursion.	<pre> public int recur(int i) { if(i % 7 == 1) return 7; return (i % 7) * recur(i + 3); } </pre>
Question 22 What is output by the line marked //q22 in the code to the right? A) 280 B) 0 C) 7 D) 70 E) There is no output due to infinite recursion.	<pre> //////////client code////////// out.println(recur(19)); //q21 out.println(recur(2)); //q22 </pre>

Question 23

What can replace **<1*>** in the code to the right so that the code compiles and runs without error?

- A) implements Comparable<Num>
- B) implements Comparable
- C) Nothing is required
- D) Either A or B.
- E) Either B or C.
- F) Any of the above.

Question 24

Which of the following terms apply to the `Num` class to the right?

- A) Overriding
- B) Overloading
- C) Encapsulation
- D) Both A and B
- E) All of the above
- F) None of the above

Question 25

Which of the following labeled methods in `Num` is required to be defined so that the code compiles and runs without error?

- A) //1
- B) //2
- C) //3
- D) //4
- E) //5
- F) //6
- G) B, E, and F.
- H) B and E.
- I) B, C, and E.
- J) A, B, C, D, and E.
- K) B, C, E, and F
- L) All of the above.

Question 26

What can replace **<2*>** in the code to the right so that the `ts` structure orders the numbers in decreasing order?

- A) Nothing is required
- B) `Collections.reverseOrder()`
- C) `-1`
- D) Either A or C.
- E) Any of the above.

Question 27

Assuming **<1*>** and **<2*>** are filled in correctly, and that no other changes have been made to the `Num` class, what is output by the code to the right?

- A) [4.5, 39.8, 212, 1234]
- B) [1234, 212, 39.8, 4.5]
- C) [null1234, null212, null39.8, null4.5]
- D) [null4.5, null39.8, null212, null1234]
- E) There is no output due to a compile error.
- F) There is no output due to a runtime error.

```
class Num <1*> {
    double val;
    String o;

    public Num(int n) { //1
        o += n;
        val = n;
    }

    public Num(double n) { //2
        o += n;
        val = n;
    }

    public Num(long n) { //3
        o += n;
        val = n;
    }

    public Num(float n) { //4
        o += n;
        val = n;
    }

    public int compareTo(Num n) { //5
        return Double.compare(val, n.val);
    }

    public String toString() { //6
        return o;
    }
}

//////////client code//////////
TreeSet<Num> ts = new TreeSet<Num>(<2*>);
ts.add(new Num(212L));
ts.add(new Num(1234));
ts.add(new Num(4.5f));
ts.add(new Num(39.8));
out.println(ts);
```

Question 28 Which of the following sorting algorithms is demonstrated in the code to the right? A) Tim Sort B) Bubble Sort C) Quick Sort D) Merge Sort E) Insertion Sort	<pre>ArrayList<Integer> a; a = new ArrayList<>(); for(int y = 0; y < 212; y++) a.add((int) (Math.random() * y * y)); Collections.sort(a); out.println(a);</pre>
Question 29 Which of the following is not a legal Java identifier? A) Ca\$H B) Final C) Hamb_rger D) Number1 E) All of the above are legal identifiers.	
Question 30 What is output by the line marked //q30 in the client code to the right? A) false false B) false true C) true false D) true true E) There is no output due to a compile error.	<pre>String s1 = "C3P0E5"; String s2 = "LikeStarWars"; String r = "[A-Z][0-9]?"; String o = "" + s1.matches(r); o += " " + s2.matches(r); out.println(o); //q30</pre>
Question 31 What is output by the client code to the right? A) false false B) false true C) true false D) true true E) There is no output due to a compile error.	<pre>r = "([A-Z]\\w*) (\\D\\W){2,4}"; o = "" + s1.matches(r); o += " " + s2.matches(r); out.println(o); //q31</pre>
Question 32 What is output by the code to the right? A) [2, 3, 5, 7] B) [7, 5, 3, 2] C) [5, 7] D) [3, 2] E) There is no output due to a runtime error.	<pre>List<Integer> nums = Arrays.asList(2, 3, 5, 7, 11, 13, 17, 19); List<Integer> result = nums.stream() .filter(n -> n < 10) .sorted((a, b) -> b - a) .limit(4) .skip(2) .collect(Collectors.toList()); out.println(result);</pre>
Question 33 What is the Big-$\mathcal{O}$ time complexity of the <code>mystery</code> method to the right in terms of n? A) $\mathcal{O}(n)$ B) $\mathcal{O}(n \log n)$ C) $\mathcal{O}(n^2)$ D) $\mathcal{O}(\log n)$ E) $\mathcal{O}(2^n)$	<pre>void mystery(int n) { for (int i = 0; i < n; i += 2) { for (int j = 0; j < i; j += 4) { out.print(j + " "); } } }</pre>
Question 34 Which of the following is the 8-bit extended two's complement representation of the number to the right? A) 1111_1101 B) 1101_1011 C) 1110_1101 D) 1101_1110 E) 1101_1101	-37_{10}

Question 35

What is output by the code to the right?

- A) 9000
- B) 60000
- C) 90000
- D) 120000
- E) There is no output due to an error.

```
enum ChronoUnit {
    MINUTES, HOURS
}

public static void main(String[] args) {
    int ms = 0;
    float time = 1.5f;
    ChronoUnit unit = ChronoUnit.MINUTES;
    switch (unit) {
        case MINUTES:
            int seconds = (int) (time * 60);
            ms = seconds * 1000;
            break;

        case HOURS:
            int minutes = (int) (time * 60);
            int seconds = minutes * 60;
            ms = seconds * 1000;
            break;
    }
    out.println(ms);
}
```

Question 36

What is output by the code to the right?

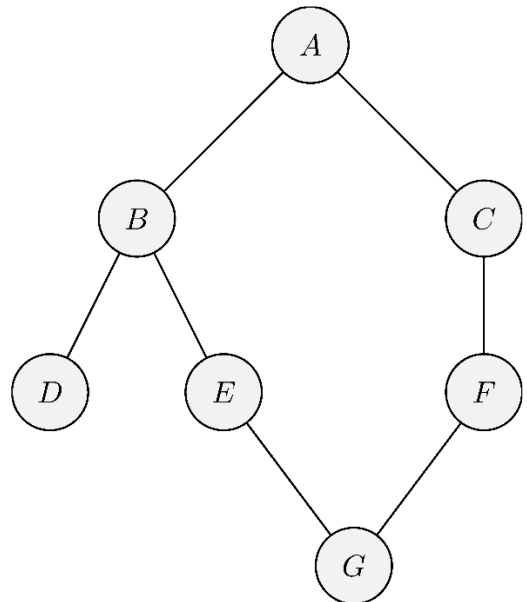
- A) Monday
- B) Wednesday
- C) Sunday
- D) Invalid day
- E) There is no output due to an error.

```
int day = 3;
String dayName = switch (day) {
    case 1 -> "Monday";
    case 2 -> "Tuesday";
    case 3 -> "Wednesday";
    case 4 -> "Thursday";
    case 5 -> "Friday";
    case 6 -> "Saturday";
    case 7 -> "Sunday";
    default -> "Invalid day";
};
out.println(dayName);
```

Question 37

Which of the following is **not** a possible BFS traversal of the graph to the right? Assume that the BFS began at the first node listed in the answer choice. For example, if the answer is *ABC*, then the first node visited is *A*.

- A) *ABCDEFGG*
- B) *BDAECGF*
- C) *BEADCGF*
- D) *FCGAEBD*
- E) *EBGADFC*

**Question 38**

Which of the following is **not** a possible DFS traversal of the graph to the right? Assume that the DFS began at the first node listed in the answer choice. For example, if the answer is *ABC*, then the first node visited is *A*.

- A) *ABDEGCF*
- B) *DBACFGE*
- C) *ABDEGFC*
- D) *FCABDEG*
- E) *EBACFGD*

Question 39

Consider the postfix expression to the right. What is the result when this expression is evaluated? Write your answer in the blank provided on your answer sheet for this question.

$$5\ 3\ +\ 8\ 2\ /\ * \ 6\ -$$
Question 40

How many permutations of `true` and `false` values for the four variables in the expression to the right resolve to `true`? Write your answer in the blank provided on your answer sheet for this question.

$$(A \cdot \bar{B} + C) \oplus (\bar{A} + D) \cdot B$$

★ ANSWER KEY – CONFIDENTIAL ★

UIL COMPUTER SCIENCE – 2025-2026 REGION

Questions (+6 points for each correct answer, -2 points for each incorrect answer)

- | | | | |
|------------------|------------------|------------------|------------------------|
| 1) <u> B </u> | 11) <u> D </u> | 21) <u> C </u> | 31) <u> D </u> |
| 2) <u> D </u> | 12) <u> B </u> | 22) <u> D </u> | 32) <u> D </u> |
| 3) <u> B </u> | 13) <u> A </u> | 23) <u> A </u> | 33) <u> C </u> |
| 4) <u> E </u> | 14) <u> C </u> | 24) <u> D </u> | 34) <u> B </u> |
| 5) <u> B </u> | 15) <u> A </u> | 25) <u> H </u> | 35) <u> E </u> |
| 6) <u> A </u> | 16) <u> A </u> | 26) <u> B </u> | 36) <u> B </u> |
| 7) <u> D </u> | 17) <u> A </u> | 27) <u> C </u> | 37) <u> C </u> |
| 8) <u> D </u> | 18) <u> B </u> | 28) <u> A </u> | 38) <u> A </u> |
| 9) <u> E </u> | 19) <u> A </u> | 29) <u> E </u> | *39) <u> 26 </u> |
| 10) <u> A </u> | 20) <u> E </u> | 30) <u> C </u> | *40) <u> 10 </u> |

* See "Explanation" section below for alternate, acceptable answers.

Note: Correct responses are based on **Java SE Development Kit 25 (JDK 25)** from Sun Microsystems, Inc. All provided code segments are intended to be syntactically correct, unless otherwise stated (e.g., "error" is an answer choice) and any necessary Java SE 25 Standard Packages have been imported. Ignore any typographical errors and assume any undefined variables are defined as used.

Explanations:

1.	B	Base 4 is the only one that lands in the [60,150] window.
2.	D	<code>a * c</code> gives 24, dividing by double <code>b</code> promotes it to 8.0, <code>10 % 4</code> gives 2, and <code>(int) (3.0 + 1.5)</code> truncates 4.5 to 4, giving <code>8.0 + 2 - 4 = 6.0</code>
3.	B	<code>println(s)</code> prints <code>hi</code> and <code>there</code> as lines 1 and 2, respectively (the <code>\n</code> inside the string is real, and <code>println</code> adds another newline after). <code>printf("%s\n%s\n", "one", "two\nthree")</code> produces one (line 3), two (line 4), and three (line 5) since the <code>\n</code> embedded in <code>"two\nthree"</code> is also a real newline. <code>print("a\n\nb")</code> gives <code>a</code> (line 6), a blank line (line 7), and <code>b</code> (starting line 8) with no trailing newline. <code>println("x\ny")</code> appends <code>x</code> to line 8 making it <code>bx</code> , then <code>y</code> becomes line 9, with <code>println</code> adding a newline. Finally <code>printf("end%s", "\n")</code> prints <code>end</code> as line 10 with a trailing newline – giving 10 total.
4.	E	<code>repeat()</code> duplicates the given string a provided number of times. The rest is simple concatenation.
5.	B	<code>(a ^ b)</code> is <code>true XOR false = true</code> , <code>!(b c)</code> is <code>!(false true) = !true = false</code> , so the left side of <code> </code> is <code>true && false = false</code> , but the right side <code>(a && c)</code> is <code>true && true = true</code> , and since <code>false true = true</code> , result is <code>true</code> .
6.	A	<code>Math.abs(-7)</code> is 7, <code>Math.round(3.6)</code> rounds to 4, so <code>Math.max(7, 4)</code> is 7, then <code>Math.floor(3.6)</code> is 3.0 and <code>Math.ceil(3.6)</code> is 4.0, so <code>Math.min(3.0, 4.0)</code> is 3.0, printing <code>7 3.0</code> .
7.	D	<code>a & b</code> is <code>0110 & 0011 = 0010 = 2</code> , so <code>flag = (2 > 1) = true</code> ; <code>'C'</code> has ASCII value 67 = <code>0100_0011</code> , and <code>& 0x0F</code> masks to the lower 4 bits giving <code>0000_0011 = 3</code> ; since <code>flag</code> is <code>true</code> the ternary picks <code>a ^ b = 0110 ^ 0011 = 0101 = 5</code> ; finally <code>3 5 = 0011 0101 = 0111 = 7</code> . The main traps are forgetting that <code>char</code> has an ASCII value that can be used in bitwise ops, and that <code>& 0x0F</code> is a common masking trick to grab the lower nibble.
8.	D	<code> </code> forces both sides to evaluate so <code>y++</code> fires making <code>y=501</code> which prints, then <code>-x--</code> evaluates as <code>-(100) = -100</code> while decrementing <code>x</code> to 99, and <code>~99 = -100</code> making <code>-100 > -100</code> false so nothing else prints – giving a single line of output: <code>501</code> .
9.	E	The total variable cannot be resolved outside of the <code>for</code> loop scope where it is defined.
10.	A	<code>arr[0] = 1</code> , <code>arr[arr[0]] = 2</code> . The first statement sets the first element of the array to 2. <code>arr[4] = arr[2]</code> return 3, and <code>arr[1] + 3 = 5</code> .
11.	D	<code>Object</code> is a class, not an interface. <code>Scanner</code> extends <code>Object</code> but it does not implement it.
12.	B	The values output are 900, 648, 448, 294, 180, 100, 48, 18, 4, 0.
13.	A	https://introcs.cs.princeton.edu/java/11precedence/
14.	C	Negative zero and zero are treated as equivalent.
15.	A	The largest possible sum occurs when the <code>random()</code> call equates to 0 every iteration, repeatedly removing 0 from the list. The sum is then the sum of 1 through 99 which can be computed with the formula $N(N + 1)/2 = 99 \cdot 100/2 = 4950$.
16.	A	An <code>ArrayStoreException</code> is thrown, so the <code>catch</code> block is reached before the 1 is printed, so the 2 is printed, and the 3 is always printed.
17.	A	You must define a type for the structure if you use <code><></code> , so only option A works.
18.	B	Java priority queue is a min-heap, so every <code>add</code> is followed by <code>heapify</code> , which swaps children and parent until no node is a child of a greater node, but it is not guaranteed to be fully sorted.
19.	A	The runtime of adding to a Java priority queue (min-heap) is $\mathcal{O}(\log N)$, and we run <code>add()</code> N times, so $N \cdot \mathcal{O}(\log N) = \mathcal{O}(N \log N)$. <code>remove()</code> is also $\mathcal{O}(\log N)$, but since we only run it a constant 3 times, it is $3 \cdot \mathcal{O}(\log N) = \mathcal{O}(1) \cdot \mathcal{O}(\log N) = \mathcal{O}(\log N)$, and we can ignore since $\mathcal{O}(\log N) < \mathcal{O}(N \log N)$
20.	E	Yeah, surprisingly all of these work.
21.	C	Simple recursion tracing
22.	D	Simple recursion tracing

23.	A	Because of the declaration of the <code>compareTo</code> method as <code>compareTo(Num n)</code> , we must implement <code>Comparable<Num></code> to specify that the parameter will be of type <code>Num</code> . Also, since the client code uses a <code>TreeSet</code> , all classes within are required to implement <code>Comparable</code> .
24.	D	The overriding is present in the <code>toString</code> method from the <code>Object</code> class, and the overloading is in the constructor.
25.	H	You only need to implement the <code>compareTo</code> , and the double constructor to be able to compile and run the code. The <code>toString()</code> would not work as intended, but not defining it would not cause an error.
26.	B	<code>Collections.reverseOrder()</code> is the only one that works at all.
27.	C	The numbers are in reverse order by value; reference question 26. The string begins with <code>null</code> , because the constructors say <code>o += n</code> , not <code>o = n</code> .
28.	A	<code>Collections.sort</code> uses Timsort.
29.	E	All are valid. Numbers can be present in variable names after the first letter, the <code>Final</code> is capitalized so not a keyword, and underscores and dollar signs are allowed in variable names.
30.	C	We are looking for 1 or more segments of a capital letter followed by either nothing or a digit, so the first string matches.
31.	D	We are looking for 1 or more segments of a capital letter followed by 1 or more word characters (<code>0-9a-zA-z_</code>), so both match. You can ignore the second half of the regex since the first matches both.
32.	D	Stream operations: <code>filter(n -> n < 10)</code> reduces the list to <code>[2, 3, 5, 7]</code> ; <code>sorted((a,b) -> b-a)</code> reverses to <code>[7, 5, 3, 2]</code> ; <code>limit(4)</code> keeps all 4 elements (no effect); <code>skip(2)</code> removes first 2 elements, leaving <code>[3, 2]</code> .
33.	C	Looking at the two loops, our suspicions from simply multiplying the degrees of n would make us believe that it is $O(n^2)$ (outer loop runs at most $O(n)$ times; inner, the same; their product is $O(n^2)$), but let's formally prove this mathematically... The following sum can model the number of times that the print statement will be called: $\sum_{i=0}^{n/2} \frac{2i}{4} = \frac{1}{2} \sum_{i=0}^{n/2} i$ Using the fact that this is an arithmetic sequence, we know that we can apply the following: $\sum_{i=0}^n i = \frac{n(n+1)}{2} \Rightarrow \frac{1}{2} \sum_{i=0}^{n/2} i = \frac{1}{2} \cdot \frac{\frac{n}{2}(\frac{n}{2}+1)}{2} = \frac{\frac{n}{2}(\frac{n}{2}-1)}{4} = \frac{\frac{n^2}{4} - \frac{n}{2}}{4} = \frac{\frac{n^2 - 2n}{4}}{4} = \frac{n^2 - 2n}{16}$ Abstracting away constants given that this is Big- O notation, we get that $\frac{n^2 - 2n}{16} \equiv n^2 - 2n = O(n^2) - O(n) = \boxed{O(n^2)}$
34.	B	Convert the positive to binary: $37_{10} = 0010_0101_2$ Flip all bits: $0010_0101_2 \Rightarrow 1101_1010_2$ Add one: $1101_1010_2 \Rightarrow 1101_1011_2$
35.	E	Not including the braces in the <code>case</code> is valid java syntax. However, it's the use of these braces which gives the scope to the code existing within the <code>case</code> statement. Because there is no case-specific scope, the duplicate variable <code>seconds</code> is the cause of the compile-time error.
36.	B	This is a <code>switch expression</code> (not statement), which can be used as an assignment value to any other expression. Note that a <code>switch</code> expression does not require the same use of <code>break</code> as <code>switch</code> statements.
37.	C	With a BFS, we are required to explore nodes in the order that they are added to a queue. The only one of these traversals that violates this principle is <i>BEADCGF</i> . Note that since <i>E</i> was explored before <i>A</i> , we require that <i>G</i> is explored before <i>C</i> . This is not the case in this traversal.
38.	A	With a DFS, we are required to explore nodes in the order that they are added to a stack. The only one of these traversals that violates this principle is <i>ABDEGCF</i> . Note that <i>C</i> is explored before <i>F</i> . This is an issue as the edge <i>AC</i> is only processed after the entire depth of edge <i>AB</i> is.

39.	26	Consider the table below showing the steps to evaluate: <table><tr><td>Expression</td><td>Stack</td></tr><tr><td>5</td><td>5</td></tr><tr><td>3</td><td>5 3</td></tr><tr><td>+</td><td>(5 + 3)</td></tr><tr><td>8</td><td>(5 + 3) 8</td></tr><tr><td>2</td><td>(5 + 3) 8 2</td></tr><tr><td>/</td><td>(5 + 3) (8 / 2)</td></tr><tr><td>*</td><td>((5 + 3) * (8 / 2))</td></tr><tr><td>6</td><td>((5 + 3) * (8 / 2)) 6</td></tr><tr><td>−</td><td>((5 + 3) * (8 / 2)) − 6 = 26</td></tr></table> Thus the final answer is 26	Expression	Stack	5	5	3	5 3	+	(5 + 3)	8	(5 + 3) 8	2	(5 + 3) 8 2	/	(5 + 3) (8 / 2)	*	((5 + 3) * (8 / 2))	6	((5 + 3) * (8 / 2)) 6	−	((5 + 3) * (8 / 2)) − 6 = 26																																																																	
Expression	Stack																																																																																						
5	5																																																																																						
3	5 3																																																																																						
+	(5 + 3)																																																																																						
8	(5 + 3) 8																																																																																						
2	(5 + 3) 8 2																																																																																						
/	(5 + 3) (8 / 2)																																																																																						
*	((5 + 3) * (8 / 2))																																																																																						
6	((5 + 3) * (8 / 2)) 6																																																																																						
−	((5 + 3) * (8 / 2)) − 6 = 26																																																																																						
40.	10	Consider the table below: <table><tr><td>A</td><td>B</td><td>C</td><td>D</td><td>$((A \wedge \neg B) \vee C) \wedge ((\neg A \vee D) \wedge B)$</td></tr><tr><td>F</td><td>F</td><td>F</td><td>F</td><td>F</td></tr><tr><td>F</td><td>F</td><td>F</td><td>T</td><td>F</td></tr><tr><td>F</td><td>F</td><td>T</td><td>F</td><td>T</td></tr><tr><td>F</td><td>F</td><td>T</td><td>T</td><td>T</td></tr><tr><td>F</td><td>T</td><td>F</td><td>F</td><td>T</td></tr><tr><td>F</td><td>T</td><td>F</td><td>T</td><td>T</td></tr><tr><td>F</td><td>T</td><td>T</td><td>F</td><td>F</td></tr><tr><td>F</td><td>T</td><td>T</td><td>T</td><td>F</td></tr><tr><td>T</td><td>F</td><td>F</td><td>F</td><td>T</td></tr><tr><td>T</td><td>F</td><td>F</td><td>T</td><td>T</td></tr><tr><td>T</td><td>F</td><td>T</td><td>F</td><td>T</td></tr><tr><td>T</td><td>F</td><td>T</td><td>T</td><td>T</td></tr><tr><td>T</td><td>T</td><td>F</td><td>F</td><td>F</td></tr><tr><td>T</td><td>T</td><td>F</td><td>T</td><td>T</td></tr><tr><td>T</td><td>T</td><td>T</td><td>F</td><td>T</td></tr><tr><td>T</td><td>T</td><td>T</td><td>T</td><td>F</td></tr></table> This table has a total of 10/16 values evaluated to true. So the answer is 10	A	B	C	D	$((A \wedge \neg B) \vee C) \wedge ((\neg A \vee D) \wedge B)$	F	F	F	F	F	F	F	F	T	F	F	F	T	F	T	F	F	T	T	T	F	T	F	F	T	F	T	F	T	T	F	T	T	F	F	F	T	T	T	F	T	F	F	F	T	T	F	F	T	T	T	F	T	F	T	T	F	T	T	T	T	T	F	F	F	T	T	F	T	T	T	T	T	F	T	T	T	T	T	F
A	B	C	D	$((A \wedge \neg B) \vee C) \wedge ((\neg A \vee D) \wedge B)$																																																																																			
F	F	F	F	F																																																																																			
F	F	F	T	F																																																																																			
F	F	T	F	T																																																																																			
F	F	T	T	T																																																																																			
F	T	F	F	T																																																																																			
F	T	F	T	T																																																																																			
F	T	T	F	F																																																																																			
F	T	T	T	F																																																																																			
T	F	F	F	T																																																																																			
T	F	F	T	T																																																																																			
T	F	T	F	T																																																																																			
T	F	T	T	T																																																																																			
T	T	F	F	F																																																																																			
T	T	F	T	T																																																																																			
T	T	T	F	T																																																																																			
T	T	T	T	F																																																																																			